

QSpice C-Block Components

C-Block Component Symbols

Revision Date: 2025.12.30

Caveats & Cautions

First and foremost: Do not assume that anything I say is true until you test it yourself. In particular, I have no insider information about QSpice. I've not seen the source code, don't pretend to understand all of the intricacies of the simulator, or, well, anything. Trust me at your own risk.

If I'm wrong, please let me know. If you think this is valuable information, let me know. (As Dave Plummer, ex-Microsoft developer and YouTube celebrity ([Dave's Garage](#)) says, "I'm just in this for the likes and subs.")

Note: *This is the fourteenth document in a clearly unplanned series:*

- *The first C-Block Basics document covered how the QSpice DLL interface works.*
- *The second C-Block Basics document demonstrated techniques to manage multiple schematic component instances, multi-step simulations, and shared resources.*
- *The third C-Block Basics document built on the second to demonstrate a technique to execute component code at the beginning and end of simulations.*
- *The fourth C-Block Basics document revisited the `Trunc()` function and provided a basic "canonical" use example.*
- *The fifth C-Block Basics document revisited the `MaxExtStepSize()` function with a focus on generating reliable clock signals from within the component.*
- *The sixth C-Block Basics document examined connecting and using QSpice bus wires with components.*
- *The seventh C-Block Basics document described recent QSpice changes (a new `Display()` function changes to `Trunc()`).*
- *The eighth C-Block Basics document detailed post-processing techniques and much more in painful detail.*
- *The ninth C-Block Basics document explained how to use the QSpice `EngAtof()` function.*
- *The tenth C-Block Basics document was a technical reference for the QSpice "Berkeley Sockets API."*
- *The eleventh C-Block Basics document extended the "Berkeley Sockets API" framework to overcome the limitations of template-generated code.*
- *The twelfth C-Block Basics document created multi-client "Berkeley Sockets" host servers for C++, Python, and Java.*
- *The thirteenth C-Block Basics document demonstrated a technique to exchange data directly between multiple DLLs and why you probably shouldn't do it.*

This paper discusses creating QSpice symbols for C-Block components. See my [GitHub repository here](#) for prior documents in this series.

Overview

This paper outlines a technique to create a QSpice symbol file (*.qsym) from a C-Block component. It also demonstrates using Programmable Attributes and discusses some considerations for using/sharing DLL component symbol files.

I assume that you know how to create C-Block component blocks in schematics; how to generate C-Block template code; and how to regenerate/update code for changes to component ports and String Attributes.

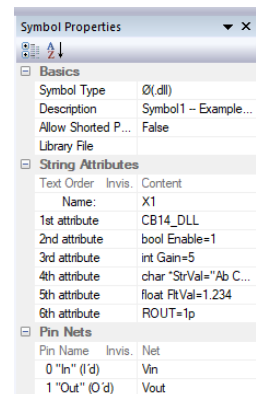
There are many ways to create symbols and I'll present only a simple way that I understand. For more techniques and details, see QSpice forum member [@KSKelvin's Github repository](#). Look for *Qspice - General Reference Guide by KSKelvin-1.pdf*.

What Is A C-Block Symbol?

Before we get into how to create and use DLL symbols, let's be clear about what they are and how they work.

A *.qsym file is merely a text file. It contains the elements that you see in the Symbol Properties pane – the bits required to generate a netlist – along with additional graphical elements. The CB14_Symbol1.qsch example from this paper looks something like this:

```
«symbol CB14_Symbol1
«type: 0(.DLL)»
«description: Symbol1 -- Example DLL Component Symbol»
«shorted pins: false»
«rect (-800,400) (800,-900) 0 0 0 0x40000000 0x40000000 -1 1 -1»
«text (0,300) 1 12 0 0x10000000 -1 -1 "X1"»
«text (0,200) 0.681 13 0 0x10000000 -1 -1 "CB14_DLL"»
«text (0,-100) 0.681 13 0 0x10000000 -1 -1 "bool Enable=1"»
«text (0,-250) 0.681 13 0 0x10000000 -1 -1 "int Gain=5"»
«text (0,-400) 0.681 13 0 0x10000000 -1 -1 "char *StrVal="Ab Cd Ef"»
«text (0,-550) 0.681 13 0 0x10000000 -1 -1 "float FltVal=1.234"»
«text (0,-700) 0.681 13 0 0x10000000 -1 -1 "ROUT=1p"»
«pin (-800,0) (0,0) 1 7 145 0x0 -1 "In"»
«pin (800,0) (0,0) 1 11 146 0x0 -1 "Out"»
»
```



If you compare the text to the Symbols Properties pane, the only thing that you don't see is the “<rect ...>” line (which defines the visible DLL block box) and graphical coordinates and codes for text alignment, font types, etc.

So, what happens when you drop this symbol into a schematic?

Well, the contents of the *.qsym file are merely copied into the schematic (which is also a text file that you can examine). That is, the symbol block in the schematic *is not a link* to the symbol file.

Changing the *.qsym file does not modify the symbol in an existing schematic in any way. Whatever was last dropped into the schematic remains unchanged. If you modify the *.qsym file, you must delete the symbol block from your schematic and add it again.

Now, on to the “how to” parts....

Step 1: Create & Test The Component

The first step is to create a schematic, add a DLL hierarchical block, generate *.cpp code, compile, and test. Revise, rinse, and repeat until the schematic, component block, and code all operate as desired. (Get this right before moving on avoid repeating later steps.)

I've already created basic examples: CB14_Test_Component.qsch and CB14_DLL.cpp.

A couple of notes:

- By default, the output impedance of DLL component ports is 1K Ω . The output resistor R1 is 1K Ω so, by default, Vout would be ½ of the voltage set on the OUT port in the DLL component. I included an ROUT “instance parameter” to change this for demonstration purposes. See QSpice Help *Simulator* → *Device Reference* → *Ø-Device* for more about the DLL component instance parameters.
- Although ROUT instance parameter appears as a String Attribute, it is not a DLL “input parameter” and not passed into the DLL. Importantly, it is the last String Attribute. *DLL String Attributes following an instance parameter are ignored.*

Once the component works properly, we can create a symbol fairly simply.

Step 2: Create The Symbol

To create the symbol file:

1. Open a new Symbol file (File → New Symbol).
2. In the schematic window, copy the DLL block to the clipboard (CTRL-C with the cursor over the *background area* in the component block).
3. Paste the clipboard into the symbol window (CTRL-V). The DLL block should appear in the symbol window.
4. Save the symbol file.

You can, of course, modify the symbol's appearance any way that you want.

- You can add/edit/delete graphical elements. For example, if you wanted the symbol to look like an op amp symbol, you could delete the gray rectangle and insert a triangle graphical element.
- You can move the pins to any desired location.
- You can add text and change font sizes, colors, etc. (Be sure that any added text elements are marked as comments lest they cause netlist errors.)
- You can “hide” String Attributes so that they appear only in the Symbol Properties pane and not in the schematic. Right-Click on the attribute in the Symbols Properties pane to change “visibility.” For example, I hid the ROUT attribute to reduce schematic clutter.
- You can even paste bitmaps into the symbol....

CB14_Symbol1.qsch is the example symbol file at this point.

Step 3: Add Symbol To A Schematic

There are a couple of ways to add a symbol to a schematic:

1. From Windows File Explorer, drag a *.qsym file onto the schematic.
2. Drag a symbol from the Symbols & IP Browser pane into the schematic.

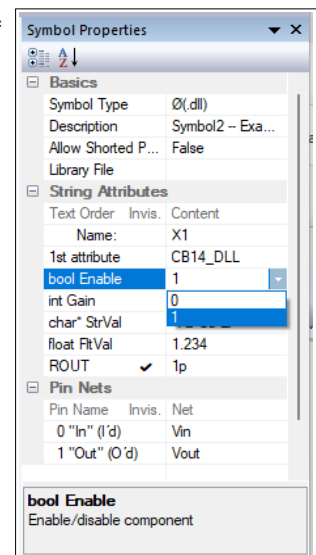
Both methods simply copy the *.qsym text into the schematic. For now, simply keep all of the files in the project folder and use method #1. We'll come back to the Symbols & IP Browser later.

CB14_Test_Symbol1.qsch is a copy of CB14_Test_Component.qsch with the DLL hierarchical block replaced with the symbol from Step #2.

Step 4: Add Programmable Attributes (Optional)

The example code currently contains boolean, integer, floating point, and char* String Attributes which are passed to the DLL. Using “Programmable Attributes,” we can:

- Move the String Attribute type (e.g., bool, int, float, char*) out of the Symbol Properties “Content” column to prevent users from accidentally changing the parameter type and breaking the interface to the DLL.
- Limit users to selecting values from a drop-down list (see the Enable parameter).
- Restrict numeric values to a minimum/maximum range (see the Gain parameter).
- Add a “user hint” to be displayed at the bottom of the Symbols Properties pane when the user selects the attribute.



This is entirely optional but useful if you plan to share your symbol with others.

CB14_Symbol2.qsym & CB14_Test_Symbol2.qsch demonstrate using Programmable Attributes. For details, see QSpice Help, *Schematic Capture* → *Symbol Editor* → *Programmable Attributes*.

Note: Currently, there is not a Programmable Attribute for string values. If a user enters a value with embedded spaces, the user must ensure that it is surrounded with double-quotes. If not properly quoted, QSpice will generate netlist errors that may or may not be clear to the user.

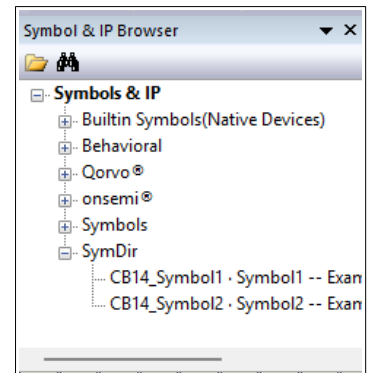
Using The Symbol & IP Browser

Note: I'll refer to the Symbol & IP Browser pane as “Symbol Browser” or “Browser pane” – this document is repetitious enough without spelling it out fully each time that I mention it.

The compelling reason to create a component symbol is to make it easy to use and easy to share. We can add it to the QSpice Symbol Browser and simply drag it onto a new schematic. Sweet.

Let's start with adding the symbol to the Symbol Browser:

1. Copy the *.qsym and *.dll files into a new folder. Let's call that folder [SymDir].
2. Create a new schematic in another empty folder. (We're simply ensuring that the *.qsym and *.dll files are not in the schematic folder.)
3. Add the [SymDir] folder to the Browser pane (Right-Click in the Browser pane, select *Add a Symbol Directory*, select the [SymDir] folder.)



Now you can drag the component symbol into the new schematic. Add some circuitry and simulation directives and you can run simulations without problems. So far, so good. But there are a couple of additional considerations....

DLL Search Paths (.libpath Directive)

When a simulation is run, QSpice looks for the DLLs in the following locations in this order:

1. If specified as an absolute path in the component, only that is used.
2. Current working directory.
3. Directories supplied in .libpath command(s).
4. QSPICE installation directory.

However, we didn't add a .libpath for our symbol folder ([SymDir]). So, how does QSpice find it?

Well, take a look at the schematic netlist. You'll find that all user-supplied folders in the Symbol Browser are automatically added as .libpath directives. This happens whenever you run a simulation regardless of whether you need/want/use Symbol Browser folders in a particular schematic.

Note also that, if folders are added/removed in the Symbol Browser, the .libpath entries will be added/removed from the netlist. If the user removes our [SymDir] – maybe while working on other projects – our component will fail with “DLL not found” errors.

A last potential issue: If there are two (or more) DLLs with the same name, QSpice might pick up the wrong one. We should probably use non-trivial names for our DLLs.

Conclusion

I hope that you find the information useful. And, of course, let me know if you find problems or suggestions for improving the code or this documentation.

--robert